

2.1.2 向量

按照面向对象思想中的数据抽象原则，可对以上的数组结构做一般性推广，使得其以上特性更具普遍性。向量（**vector**）就是线性数组的一种抽象与泛化，它也是由具有线性次序的一组元素构成的集合 $V = \{ v_0, v_1, \dots, v_{n-1} \}$ ，其中的元素分别由秩相互区分。

各元素的秩（**rank**）互异，且均为 $[0, n)$ 内的整数。具体地，若元素 e 的前驱元素共计 r 个，则其秩就是 r 。以前介绍的线性递归为例，运行过程中所出现过的所有递归实例，按照相互调用的关系可构成一个线性序列。在此序列中，各递归实例的秩反映了它们各自被创建的时间先后，每一递归实例的秩等于早于它出现的实例总数。反过来，通过 r 亦可唯一确定 $e = v_r$ 。这是向量特有的元素访问方式，称作“循秩访问”（**call-by-rank**）。

经如此抽象之后，我们不再限定同一向量中的各元素都属于同一基本类型，它们本身可以是来自于更具一般性的某一类的对象。另外，各元素也不见得同时具有某一数值属性，故而并不保证它们之间能够相互比较大小。

以下首先从向量最基本的接口出发，设计并实现与之对应的向量模板类。然后在元素之间具有大小可比性的假设前提下，通过引入通用比较器或重载对应的操作符明确定义元素之间的大小判断依据，并强制要求它们按此次序排列，从而得到所谓有序向量，并介绍和分析此类向量的相关算法及其针对不同要求的各种实现版本。

§ 2.2 接口

2.2.1 ADT接口

作为一种抽象数据类型，向量对象应支持如下操作接口。

表2.1 向量ADT支持的操作接口

操 作 接 口	功 能	适 用 对 象
size()	报告向量当前的规模（元素总数）	向量
get(r)	获取秩为r的元素	向量
put(r, e)	用e替换秩为r元素的数值	向量
insert(r, e)	e作为秩为r元素插入，原后继元素依次后移	向量
remove(r)	删除秩为r的元素，返回该元素中原存放的对象	向量
disordered()	判断所有元素是否已按非降序排列	向量
sort()	调整各元素的位置，使之按非降序排列	向量
find(e)	查找等于e且秩最大的元素	向量
search(e)	查找目标元素e，返回不大于e且秩最大的元素	有序向量
deduplicate()	剔除重复元素	向量
uniquify()	剔除重复元素	有序向量
traverse()	遍历向量并统一处理所有元素，处理方法由函数对象指定	向量

以上向量操作接口，可能有多种具体的实现方式，计算复杂度也不尽相同。而在引入秩的概念并将外部接口与内部实现分离之后，无论采用何种具体的方式，符合统一外部接口规范的任一实现均可直接地相互调用和集成。

2.2.2 操作实例

按照表2.1定义的ADT接口，表2.2给出了一个整数向量从被创建开始，通过ADT接口依次实施一系列操作的过程。请留意观察，向量内部各元素秩的逐步变化过程。

表2.2 向量操作实例

操作	输出	向量组成（自左向右）	操作	输出	向量组成（自左向右）
初始化			disordered()	3	4 3 7 4 9 6
insert(0, 9)		9	find(9)	4	4 3 7 4 9 6
insert(0, 4)		4 9	find(5)	-1	4 3 7 4 9 6
insert(1, 5)		4 5 9	sort()		3 4 4 6 7 9
put(1, 2)		4 2 9	disordered()	0	3 4 4 6 7 9
get(2)	9	4 2 9	search(1)	-1	3 4 4 6 7 9
insert(3, 6)		4 2 9 6	search(4)	2	3 4 4 6 7 9
insert(1, 7)		4 7 2 9 6	search(8)	4	3 4 4 6 7 9
remove(2)	2	4 7 9 6	search(9)	5	3 4 4 6 7 9
insert(1, 3)		4 3 7 9 6	search(10)	5	3 4 4 6 7 9
insert(3, 4)		4 3 7 4 9 6	uniquify()		3 4 6 7 9
size()	6	4 3 7 4 9 6	search(9)	4	3 4 6 7 9

2.2.3 Vector模板类

按照表2.1确定的向量ADT接口，可定义Vector模板类如代码2.1所示。

```
1 typedef int Rank; //秩
2 #define DEFAULT_CAPACITY 3 //默认的初始容量（实际应用中可设置为更大）
3
4 template <typename T> class Vector { //向量模板类
5 protected:
6     Rank _size; int _capacity; T* _elem; //规模、容量、数据区
7     void copyFrom(T const* A, Rank lo, Rank hi); //复制数组区间A[lo, hi)
8     void expand(); //空间不足时扩容
9     void shrink(); //装填因子过小时压缩
10    bool bubble(Rank lo, Rank hi); //扫描交换
11    void bubbleSort(Rank lo, Rank hi); //起泡排序算法
12    Rank max(Rank lo, Rank hi); //选取最大元素
13    void selectionSort(Rank lo, Rank hi); //选择排序算法
14    void merge(Rank lo, Rank mi, Rank hi); //归并算法
15    void mergeSort(Rank lo, Rank hi); //归并排序算法
16    Rank partition(Rank lo, Rank hi); //轴点构造算法
17    void quickSort(Rank lo, Rank hi); //快速排序算法
18    void heapSort(Rank lo, Rank hi); //堆排序（稍后结合完全堆讲解）
19 public:
```

```

20 // 构造函数
21 Vector(int c = DEFAULT_CAPACITY, int s = 0, T v = 0) //容量为c、规模为s、所有元素初始为v
22 { _elem = new T[_capacity = c]; for (_size = 0; _size < s; _elem[_size++] = v); } //s <= c
23 Vector(T const* A, Rank lo, Rank hi) { copyFrom(A, lo, hi); } //数组区间复制
24 Vector(T const* A, Rank n) { copyFrom(A, 0, n); } //数组整体复制
25 Vector(Vector<T> const& V, Rank lo, Rank hi) { copyFrom(V._elem, lo, hi); } //向量区间复制
26 Vector(Vector<T> const& V) { copyFrom(V._elem, 0, V._size); } //向量整体复制
27 // 析构函数
28 ~Vector() { delete [] _elem; } //释放内部空间
29 // 只读访问接口
30 Rank size() const { return _size; } //规模
31 bool empty() const { return !_size; } //判空
32 int disordered() const; //判断向量是否已排序
33 Rank find(T const& e) const { return find(e, 0, (Rank)_size); } //无序向量整体查找
34 Rank find(T const& e, Rank lo, Rank hi) const; //无序向量区间查找
35 Rank search(T const& e) const //有序向量整体查找
36 { return (0 >= _size) ? -1 : search(e, (Rank)0, (Rank)_size); }
37 Rank search(T const& e, Rank lo, Rank hi) const; //有序向量区间查找
38 // 可写访问接口
39 T& operator[](Rank r) const; //重载下标操作符，可以类似于数组形式引用各元素
40 Vector<T> & operator=(Vector<T> const&); //重载赋值操作符，以便直接克隆向量
41 T remove(Rank r); //删除秩为r的元素
42 int remove(Rank lo, Rank hi); //删除秩在区间[lo, hi)之内的元素
43 Rank insert(Rank r, T const& e); //插入元素
44 Rank insert(T const& e) { return insert(_size, e); } //默认作为末元素插入
45 void sort(Rank lo, Rank hi); //对[lo, hi)排序
46 void sort() { sort(0, _size); } //整体排序
47 void unsort(Rank lo, Rank hi); //对[lo, hi)置乱
48 void unsort() { unsort(0, _size); } //整体置乱
49 int deduplicate(); //无序去重
50 int uniquify(); //有序去重
51 // 遍历
52 void traverse(void (*)(T&)); //遍历（使用函数指针，只读或局部性修改）
53 template <typename VST> void traverse(VST&); //遍历（使用函数对象，可全局性修改）
54 }; //Vector

```

代码2.1 向量模板类Vector

这里通过模板参数T，指定向量元素的类型。于是，以Vector<int>或Vector<float>之类的形式，可便捷地引入存放整数或浮点数的向量；而以Vector<Vector<char>>之类的形式，则可直接定义存放字符的二维向量等。这一技巧有利于提高数据结构选用的灵活性和运行效率，并减少出错，因此将在本书中频繁使用。

在表2.1所列基本操作接口的基础上，这里还扩充了一些接口。比如，基于size()直接实现

的判空接口`empty()`，以及区间删除接口`remove(lo, hi)`、区间查找接口`find(e, lo, hi)`等。它们多为上述基本接口的扩展或变型，可使代码更为简洁易读，并提高计算效率。

这里还提供了`sort()`接口，以将向量转化为有序向量。为此可有多种排序算法供选用，本章及后续章节，将陆续介绍它们的原理、实现并分析其效率。排序之后，向量的很多操作都可更加高效地完成，其中最基本和最常用的莫过于查找。因此，这里还针对有序向量提供了`search()`接口，并将详细介绍若干种相关的算法。为便于对`sort()`算法的测试，这里还设有一个`unsort()`接口，以将向量随机置乱。在讨论这些接口之前，我们首先介绍基本接口的实现。

§ 2.3 构造与析构

由代码2.1可见，向量结构在内部维护一个元素类型为`T`的私有数组`_elem[]`：其容量由私有变量`_capacity`指示；有效元素的数量（即向量当前的实际规模），则由`_size`指示。此外还进一步地约定，在向量元素的秩、数组单元的逻辑编号以及物理地址之间，具有如下对应关系：

向量中秩为`r`的元素，对应于内部数组中的`_elem[r]`，其物理地址为`_elem + r`

因此，向量对象的构造与析构，将主要围绕这些私有变量和数据区的初始化与销毁展开。

2.3.1 默认构造方法

与所有的对象一样，向量在使用之前也需首先被系统创建——借助构造函数（`constructor`）做初始化（`initialization`）。由代码2.1可见，这里为向量重载了多个构造函数。

其中默认的构造方法是，首先根据创建者指定的初始容量，向系统申请空间，以创建内部私有数组`_elem[]`；若容量未明确指定，则使用默认值`DEFAULT_CAPACITY`。接下来，鉴于初生的向量尚不包含任何元素，故将指示规模的变量`_size`初始化为0。

整个过程顺序进行，没有任何迭代，故若忽略用于分配数组空间的时间，共需常数时间。

2.3.2 基于复制的构造方法

向量的另一典型创建方式，是以某个已有的向量或数组为蓝本，进行（局部或整体的）克隆。代码2.1中虽为此功能重载了多个接口，但无论是已封装的向量或未封装的数组，无论是整体还是区间，在入口参数合法的前提下，都可归于如代码2.2所示的统一的`copyFrom()`方法：

```
1 template <typename T> //元素类型
2 void Vector<T>::copyFrom(T const* A, Rank lo, Rank hi) { //以数组区间A[lo, hi)为蓝本复制向量
3     _elem = new T[_capacity = 2 * (hi - lo)]; _size = 0; //分配空间，规模清零
4     while (lo < hi) //A[lo, hi)内的元素逐一
5         _elem[_size++] = A[lo++]; //复制至_elem[0, hi - lo)
6 }
```

代码2.2 基于复制的向量构造器

`copyFrom()`首先根据待复制区间的边界，换算出新向量的初始规模；再以双倍的容量，为内部数组`_elem[]`申请空间。最后通过一趟迭代，完成区间`A[lo, hi)`内各元素的顺次复制。

若忽略开辟新空间所需的时间，运行时间应正比于区间宽度，即 $O(hi - lo) = O_size)$ 。